DUAL V2.2 / ROBOT SOUL / ARCHITECTURE REVIEW

Agent + Skill 架構梳理

Agent + Skill Architecture Review

從「VLM 直接調用函數」到「Plan → Approval → Task → Executor」的受監督機器人運行時。

From direct VLM function calling to a supervised robot runtime built around Plan → Approval → Task → Executor.

當前實現 / Current implementation: dual_v2_2_agent

審查日期 / Review date: 2026-08-27

EXECUTIVE SUMMARY

核心判斷 / Executive Assessment

繁體中文

新架構的本質不是「換了一個模型」。它重新定義了模型、網頁、Agent、Skill、Tool 與底層控制之間的權限邊界。

傳統 VLM + Functions 模式讓模型選擇並調用函數，規劃和執行容易發生在同一鏈路中。當前實現讓模型或規則 Planner 只產生高層語義 Plan；Plan 本身不運動機器人，必須經過白名單校驗、現場檢查和逐 Plan 批准，才會創建 Hardware Task。

Python Agent 是機械臂、相機與感知模型的唯一軟件所有者。Skill 負責完整任務策略，Tool 負責原子能力，RobotRAM 繼續承擔已驗證的感知、規劃與執行。

最大收益：安全權威分離、任務可觀測、能力可復用、故障可恢復。

主要代價：進程和狀態更多、策略存在雙份校驗、Planner 泛化能力仍有限。

ENGLISH

The new architecture is not merely a model replacement. It reallocates authority across the model, UI, Agent, Skills, Tools, and control stack.

In a conventional VLM + Functions design, the model chooses functions and planning may flow directly into dispatch. The current implementation allows a rule-based or LLM Planner to propose only a high-level semantic Plan. A Plan is inert until allowlist validation, on-site checks, and per-Plan approval create a Hardware Task.

The Python Agent is the sole software owner of arms, cameras, and perception models. Skills own task strategy, Tools expose atomic capabilities, and RobotRAM continues to provide the validated perception, planning, and execution stack.

Primary gains: separated safety authority, observability, reuse, and recovery.

Main costs: more processes and state, duplicated policy checks, and limited planner generality.

指標 / Metric	當前實現 / Current implementation
高層能力	5 個內建 Skills；其中 4 個進入網頁/Harness 真機執行白名單。
原子能力	3 個感知 Tools + 5 個操作 Tools。
硬件所有權	1 個 Python Agent 持有機械臂、相機與感知模型。
啓動模式	start：直接真機；console：網頁在線、硬件 offline，現場一鍵啓動。

LAYERED ARCHITECTURE

層次結構 / System Layers

層 / Layer	實現 / Implementation	職責 / Responsibility
01 操作入口	Robot Soul GUI + Harness	目標輸入、Plan 審查、現場審批、Task Timeline。 Interfaces for goals, review, approval, and status.
02 策略與審批	TypeScript + Python gates	Skill 白名單、參數、動作預算、Plan TTL、checklist 與確認口令。
03 Agent Server	robot_agent/server.py	Plan/Task、硬件鎖、異步執行、事件、取消與審批請求。
04 Planner	robot_agent/planner.py	規則優先、可選 LLM；自然語言 → SkillCall Plan。
05 Executor	robot_agent/executor.py	依賴校驗、執行時語義解析、逐 Skill 執行與事件發送。
06 Skill	robot_agent/skills/	完整任務策略：感知、抓放、整理、恢復和啓動。
07 Tool	robot_agent/tools/	3 個感知 + 5 個操作元動作；統一返回 ToolResult。
08 身體執行層	RobotRAM + CuRobo/Upper/RT	Step1 – 5、軌跡規劃、UDP 控制與物理硬件。

繁體中文職責邊界

- Plan：純數據，包含 instruction、SkillCall 與 constraints。
- Skill：任務級組合，封裝順序、前置條件、失敗恢復與業務語義。
- Tool：確定性原子能力，是進入 RobotRAM 的受控適配層。
- RobotContext：持有相機、機械臂、模型、標定和當前場景狀態。
- RobotRAM：底層能力被復用，而非被重新實現。

ENGLISH RESPONSIBILITIES

- Plan: inert data containing instruction, SkillCalls, and constraints.
- Skill: task composition owning order, preconditions, recovery, and semantics.
- Tool: deterministic atomic capability and controlled RobotRAM adapter.
- RobotContext: owns arms, cameras, models, calibration, and scene state.
- RobotRAM: existing low-level capabilities are reused, not rewritten.

關鍵不變量：模型不能提交關節角、CAN、shell、任意坐標或任意原子調用。

Key invariant: the model cannot submit joints, CAN, shell, arbitrary poses, or arbitrary atomic calls.

EXECUTION FLOW

真實執行鏈路 / End-to-End Execution

階段 / Stage	繁體中文說明	English
1 Goal	用戶輸入「整理桌面」。	The operator submits 「Clean the table」.
2 Plan	Planner 生成 tidy_table SkillCall；不初始化硬件。	The Planner emits a tidy_table SkillCall without hardware bootstrap.
3 Approval	校驗白名單、TTL、現場 checklist 與確認口令。	Validate allowlist, TTL, on-site checklist, and confirmation phrase.
4 Task	Server 創建唯一活動 Task，並獲取 hardware lock。	The Server creates the sole active Task and acquires the hardware lock.
5 Perception	Ground → Estimate → Localize，建立當前場景。	Ground → Estimate → Localize builds the current scene.
6 Skill	tidy_table 過濾目標，逐項調用 pick_and_place。	tidy_table filters candidates and invokes pick_and_place.
7 Tools	Approach、Grasp、Transport、Place、Reset。	Atomic approach, grasp, transport, place, and reset operations.
8 Hardware	Step5 → RobotArm UDP → CuRobo / RT Control。	Step5 → RobotArm UDP → CuRobo / RT Control.

繁體中文

執行時解析物體：Plan 使用 {category, color} 語義引用。感知後，Executor 才解析為 object_2 等臨時 ID，因此 Plan 不綁定某一檢測幀。

整理桌面展開：Planner 只提交一個 tidy_table Skill。Skill 感知場景、過濾缺失物體，再對碗、麵包和杯子逐項調用 pick_and_place。

錯誤流：ToolResult → Skill → Executor → Server Task → 網頁 Timeline，可指出物體、目標、動作、手臂、位置與底層反饋。

ENGLISH

Execution-time object resolution: Plans carry {category, color}; after perception, the Executor resolves transient object IDs, keeping Plans frame-independent.

Tidying expansion: the Planner emits one tidy_table Skill. The Skill perceives, filters unavailable objects, and invokes pick_and_place for bowl, bread, and cup.

Error flow: ToolResult → Skill → Executor → Server Task → Web Timeline, preserving object, target, action, arm, pose, and low-level feedback.

Plan 不是 Task；Task 也不是直接的原子動作。每一層都進一步收窄權限並增加驗證。

A Plan is not a Task, and a Task is not a raw atomic command. Each layer narrows authority and adds validation.

ARCHITECTURE COMPARISON

VLM + Functions 與 Agent + Skill 對比

維度 / Dimension	傳統 VLM + Functions	當前 Agent + Skill	價值 / Impact
模型輸出	直接選擇 approach / grasp / place 等函數。	只提交高層 Skill Plan。	意圖與確定性運動分離。 Intent is separated from deterministic motion.
執行觸發	推理後可直接 dispatch。	Plan → 審查 → 批准 → Task。	Plan 可拒絕、過期、審計，並且本身不運動。
硬件所有權	多個腳本或 bridge 可能進入硬件。	Python Agent 是唯一硬件所有者。	避免相機、機械臂、GPU 和控制狀態衝突。
能力抽象	函數薄包裝，流程散落在 prompt 或腳本。	Skill 封裝策略；Tool 封裝原子動作。	能力可測試、組合、版本化和授權。
物體引用	較早綁定 object_0 / object_1。	語義引用在感知後解析。	Plan 與臨時場景編號解耦。
安全邊界	依賴 prompt 與操作紀律。	白名單、參數、鎖、預算和現場批准。	安全由代碼實施，不依賴模型自覺。
可觀測性	以日誌為主，上下文易丟失。	Plan、Task、step、placement、operation 事件鏈。	能定位具體物體與失敗環節。
遷移成本	直接運行 RobotRAM。	RobotRAM 被 Tool/Skill 包裝後復用。	保留已驗證能力，避免推倒重寫。

架構意圖：從「VLM/LLM 作為遙控器」轉向「模型負責意圖，Agent 負責狀態與執行，人類負責逐 Plan 批准」。

Architectural intent: move from 「the model as remote control」 to 「model for intent, Agent for state/execution, human for per-Plan approval.」

LIFECYCLE & SUPERVISION

launch_robot_soul.sh 增加的生命週期能力

能力 / Capability	繁體中文	English
統一裝配	預檢、構建 Harness Web、準備配置、共享 token，並按順序啓動硬件面、Harness、GUI 與 Supervisor。	Preflight, web build, configuration, shared token, and ordered startup.
雙模式	start 直接啓動真機；console 只啓動網頁/Harness，硬件保持 offline。	start boots hardware; console keeps hardware offline until on-site startup.
顯式狀態機	offline → starting → healthy / faulted → recovering。	UI availability and hardware readiness become separate states.
故障域分離	GUI/Harness 失敗觸發完整停機；硬件失敗則鎖定硬件並保留網頁。	UI failure stops all; hardware failure preserves diagnostics while locking motion.
現場恢復	網頁啓動與恢復要求確認口令、生命週期鎖和 sudo 授權。	Startup and recovery require confirmation, lifecycle locking, and sudo authorization.
統一退出	按順序清理 Agent、Planner、Upper 與 RT，減少殘留進程。	Coordinated shutdown reduces stale processes and ownership conflicts.

繁體中文判斷

launch_robot_soul.sh 不只是啓動腳本。它把認知面、操作面與硬件面裝配為具有健康檢查、故障鎖定和人工恢復能力的產品運行時。

- Tier 1：Harness 或統一網頁連續失敗，執行完整安全停機。
- Tier 2：硬件面連續失敗，停止並鎖定硬件，保留 Harness 與網頁。
- Console 模式允許先啓動操作台，再由現場人員一鍵初始化真機棧。

ENGLISH ASSESSMENT

launch_robot_soul.sh is more than a startup script. It assembles the cognitive, operator, and hardware planes into a supervised product runtime.

- Tier 1: repeated Harness or unified UI failure causes a complete safe shutdown.
- Tier 2: repeated hardware failure locks and stops hardware while preserving diagnostics.
- Console mode allows the operator surface to start before on-site hardware initialization.

STRENGTHS & CONSTRAINTS

優勢與當前限制 / Strengths and Limits

優勢 / STRENGTHS

- 安全權威分離：模型提議、操作員批准、Agent 執行、控制棧驅動。
- 可觀測：Plan ID、Task 狀態、階段事件與詳細失敗上下文。
- 可測試：Planner、Skill、策略、審批、Origin 與狀態機可獨立驗證。
- 可復用：pick_and_place 可被單任務與 tidy_table 組合。
- 漸進遷移：保留 RobotRAM Step1 – 5 與控制棧。
- 故障可恢復：硬件 fault 不必殺死網頁和 Harness。

English: separated authority, observability, testability, reusable Skills, incremental migration, and isolated recovery.

限制 / LIMITS

- Planner 較窄：當前主要覆蓋整理與單次抓放。
- 整理仍有配方：按感知過濾，但不是完全動態關係推導。
- 感知誤檢：有效 mask/位姿仍可能對應錯誤語義。
- 場景可能過期：移動物體後需要更新 world state 或重新感知。
- 狀態主要在內存：重啓會丟失短期 Plan、審批與歷史。
- 策略雙份維護：TS 與 Python 白名單、預算可能漂移。
- 單 Task 串行：暫無隊列、優先級和調度。
- 取消不是急停：cancel_event 不能替代實體急停。

English: narrow planning, recipe-driven tidying, false positives, stale state, in-memory lifecycle, policy drift, and no queue.

直接結論：當前系統還不是「通用自主機器人 Agent」；它是一個安全、可審查、可擴展的任務運行時，Planner 與世界模型仍處於早期階段。

Direct conclusion: this is not yet a general autonomous robot. It is a safe and extensible task runtime with an early-stage planner and world model.

ROADMAP

演進潛力 / Development Potential

短期 / SHORT TERM

1. 使用單一 Skill Schema 生成 TS/Python 校驗與動作預算。
   Generate both policy layers from one Skill Schema.
2. 每次放置後更新世界模型；被移動目標強制重新感知。
   Refresh world state after each placement.
3. 完整保留 operation、arm、action、pose 等結構化事件。
   Preserve structured operation metadata.
4. 持久化 Plan、Task、審批、事件和執行結果。
   Persist Plans, Tasks, approvals, events, and results.

中期 / MID TERM

1. 為 Skill 增加前置條件、後置條件、風險等級、版本與重試。
   Add pre/postconditions, risk, versions, and retries.
2. 將整理任務升級為場景驅動的動態 Skill DAG。
   Generate scene-driven Skill DAGs.
3. 加入仿真回放、碰撞預檢和 Plan Diff。
   Add simulation replay, collision preflight, and Plan Diff.
4. 建立有限 Task 隊列，同時保持硬件 Executor 串行。
   Queue Tasks while preserving serial hardware execution.

長期 / LONG TERM

1. 使用受 Skill Manifest 約束的 LLM Planner。
   Use manifest-constrained LLM planning.
2. 多個只規劃 Agent 並行提出方案，由單一執行 Agent 仲裁。
   Allow many planners but one execution authority.
3. 建立跨機器人 Capability Profile 與版本兼容。
   Build cross-robot capability profiles.
4. 用執行結果與視覺復核形成閉環學習和主動恢復。
   Learn from outcomes and visual verification.

下一階段不應繼續增加更多「模型可直接調用的函數」，而應增加帶契約、版本、風險和恢復策略的 Skill。

The next phase should add contract-driven, versioned, risk-rated, recoverable Skills—not more functions directly exposed to the model.

CONCLUSION & SOURCE MAP

結論與關鍵入口 / Conclusion and Source Map

必須保持的架構不變量

1. Python Agent 始終是唯一硬件所有者。
2. 模型只能提交語義 Plan，不能提交關節、CAN、shell 或任意原子調用。
3. Plan 與 Task 始終分離；真機運動始終經過獨立現場批准。
4. 軟件取消與 Supervisor 不能替代獨立實體急停。

ARCHITECTURAL INVARIANTS

1. The Python Agent remains the sole hardware owner.
2. The model submits semantic Plans, never joints, CAN, shell, or arbitrary calls.
3. Plan and Task remain separate; motion always requires on-site approval.
4. Software cancellation and supervisors never replace the physical emergency stop.

路徑 / Path	職責 / Responsibility
launch_robot_soul.sh	統一裝配、start/console、Supervisor、硬件啓動與恢復。 Runtime assembly and supervision.
robot_agent/server.py	Plan、Task、審批、硬件鎖、事件與唯一硬件所有權。
robot_agent/planner.py	規則優先、可選 LLM 的自然語言到 Skill Plan。
robot_agent/executor.py	依賴校驗、語義解析與逐 Skill 執行。
robot_agent/skills/	感知、抓放、整理、復位和啓動等任務級能力。
robot_agent/tools/	感知與操作原子能力，適配 RobotRAM。
RobotRAM/scripts/step1…step5	現有感知、位姿估計、抓取規劃和執行。

最終判斷：Agent + Skill
架構最大的長期價值，是在不放棄現有機器人能力的前提下，把自然語言智能、確定性執行和物理安全變成三個可以獨立演進的系統。

Final assessment: the long-term value is the ability to evolve language intelligence, deterministic execution, and physical safety independently while preserving existing robot capabilities.